

BRIEF INSIGHTS and SUMMARY AZURE CLOUD CONCEPTS

Objective:

To understand the fundamental concepts of Microsoft Azure cloud services and gain hands-on experience in creating and managing cloud resources. The objective of this assignment is to build and execute an end-to-end data pipeline using Azure Storage Account and Azure Data Factory, including data transfer, metadata validation, access management, and pipeline monitoring.

Task 1 : Explore Azure Portal and Create a Resource Group

Azure Portal

Azure Portal (portal.azure.com) is Microsoft's official web-based interface for managing all Azure cloud services. It eliminates the need to manage physical servers, everything is handled through a clean, browser-based dashboard from anywhere in the world.

The key components of the Azure Portal are:

Dashboard - The first screen you see after logging in. It gives a customizable overview of all your active resources, recent activity, and cost summaries in one place.

Search Bar - Located at the top of the screen, it allows you to instantly navigate to any Azure service: Storage Accounts, Data Factory, Virtual Machines, and more — without manually browsing menus.

Left Sidebar - Contains pinned favorites and frequently used services like Home, Resource Groups, and All Resources, making navigation faster during repetitive tasks.

+ Create a Resource - A dedicated button that opens the Azure Marketplace, where you can browse and deploy over 100+ cloud services including Storage, Compute, Databases, and Analytics tools.

Resource Groups - A built-in organizational feature that acts like a project folder. All resources belonging to the same project are kept together, making management, monitoring, and deletion much easier.

Access Control (IAM) - Available on every resource, this section lets you assign and manage roles such as Reader, Contributor, and Owner to control who can view or modify each resource securely.

Monitoring Section - Provides real-time metrics, activity logs, diagnostic data, and configurable alerts for all deployed resources to help track performance and detect issues early.

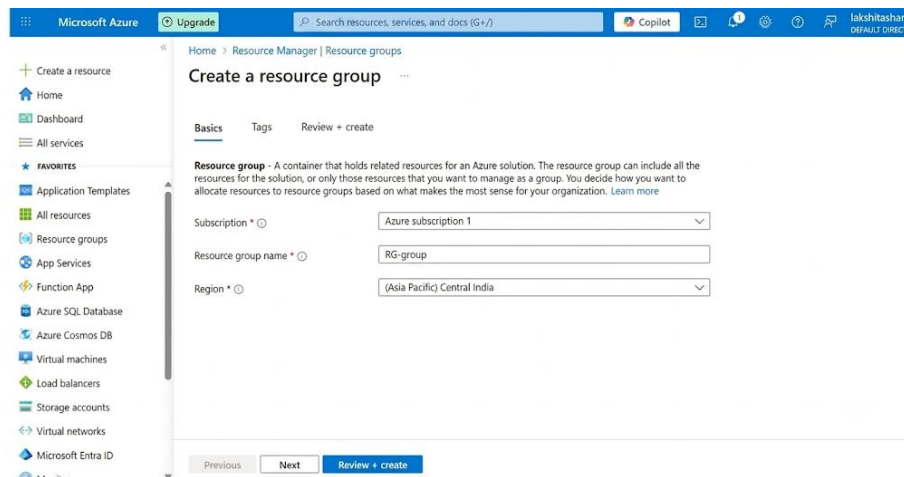
Notifications (Bell Icon) - Located at the top right, it shows instant feedback on deployment operations whether a resource was created successfully or if an error occurred.

Resource Group

A Resource Group is a logical container in Azure that groups all related resources for a project together. Instead of having your Storage Account, Data Factory, and other services scattered randomly across your Azure account, a Resource Group keeps them organized under one name.

In this assignment, we created a Resource Group named **RG-group** to hold both the Storage Account and the Azure Data Factory instance. This makes it easy to manage everything together, monitor combined costs, set access permissions at the group level, and delete all resources at once when the project is complete.

Navigation Path: Azure Portal → Resource Groups → + Create → Fill Subscription, Name, Region → Review + Create → Create



Notifications

[More events in the activity log →](#)

[Dismiss all](#)

✓ Resource group created

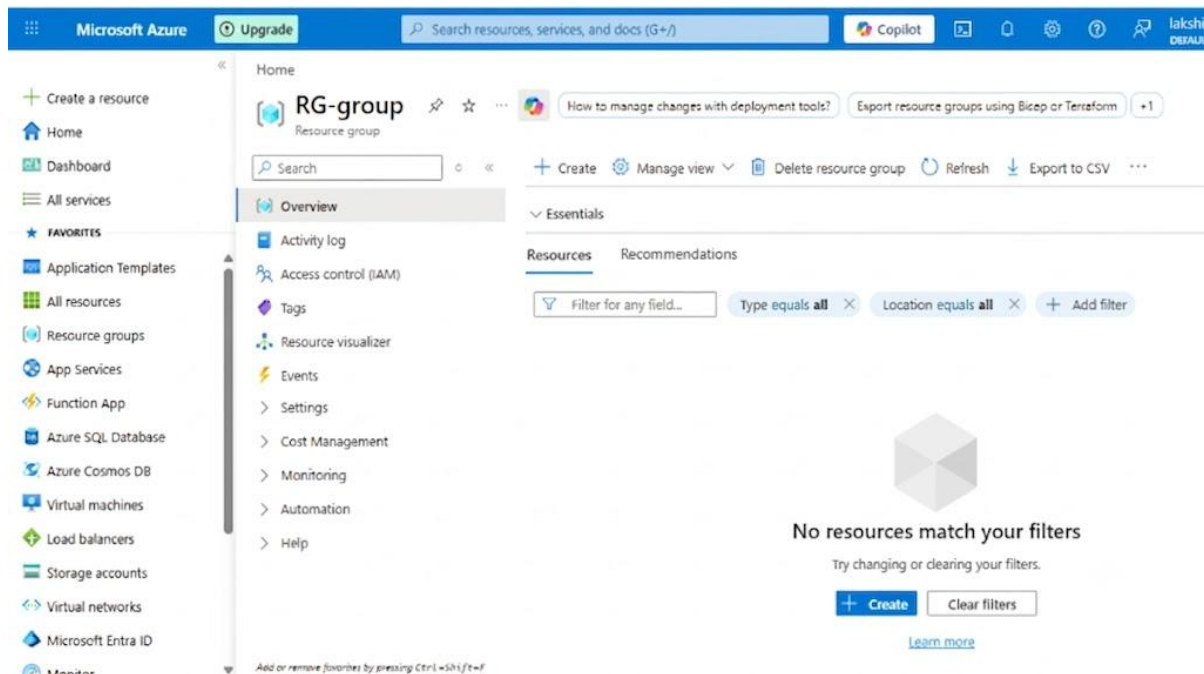
Creating resource group 'RG-group' in subscription 'Azure subscription 1' succeeded.

[Go to resource group](#)

[Pin to dashboard](#)

a few seconds ago

This page displays all resources contained within the Resource Group and serves as the main management dashboard. From here, users can create additional resources, manage display settings, monitor activities through logs, export resource information, and perform administrative actions such as deleting the group.



Task 2 - Storage Setup

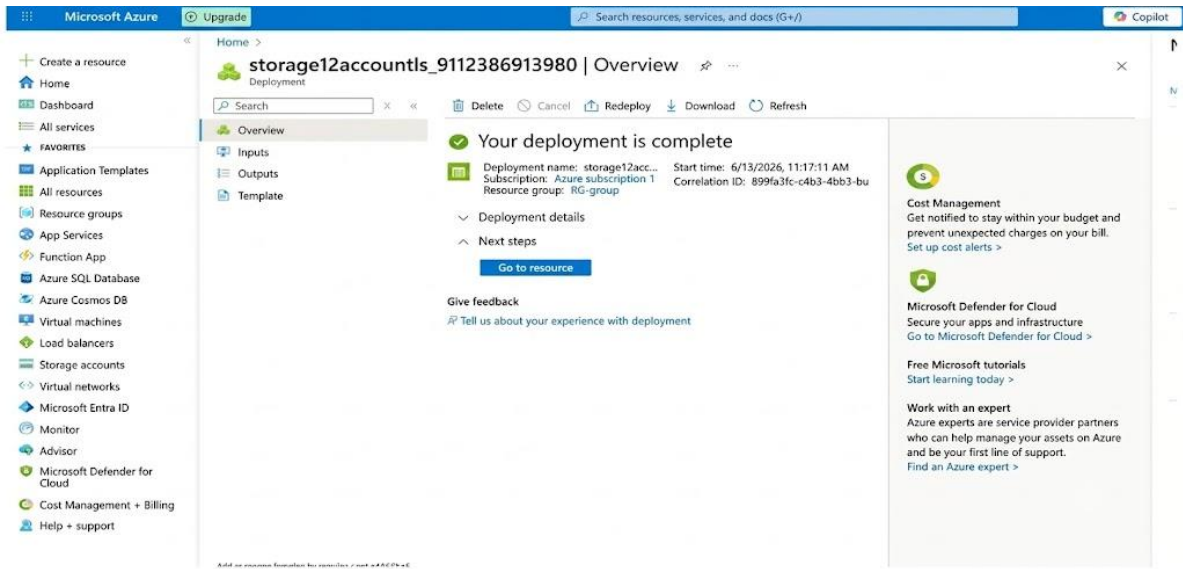
Azure Storage Account

An Azure Storage Account is Microsoft's cloud-based file storage service. It provides a highly available, secure, and scalable place to store data in the cloud — without managing any physical hardware. Think of it as a cloud hard drive that your data pipelines can read from and write to.

Steps:

Azure Portal → Storage Accounts → + Create → filled in required details → Review + Create → Create

In this task, we created a Storage Account named **storage12accountls** to serve as the data store for our pipeline. Once the deployment was complete, the portal confirmed with the message "Your deployment is complete."



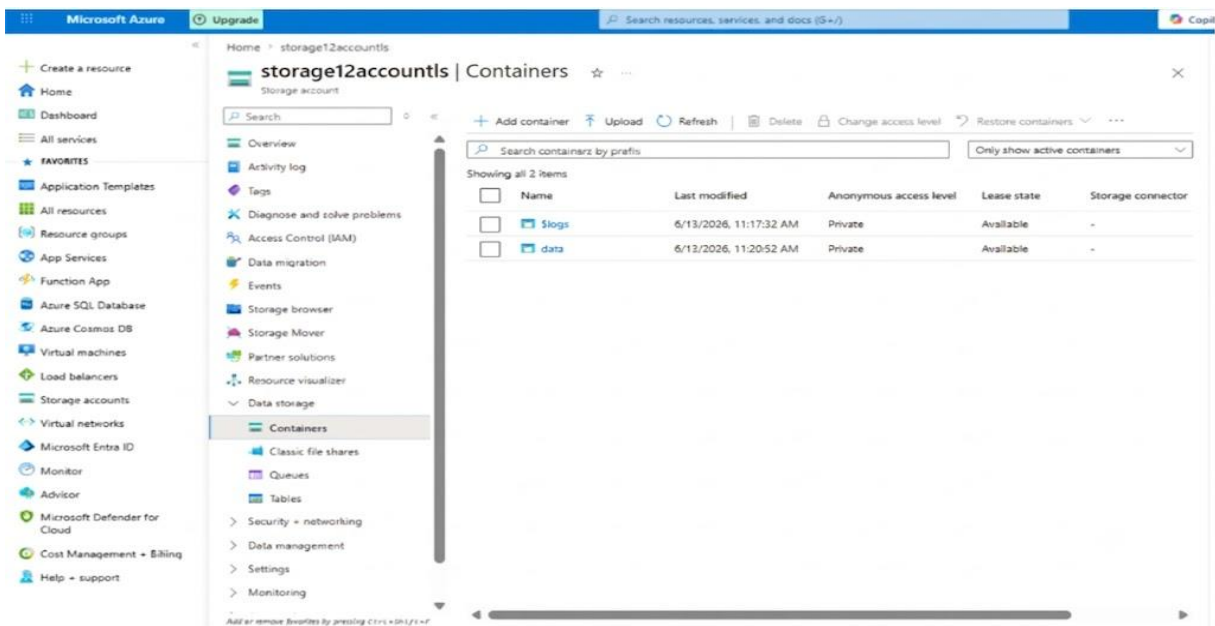
Blob Container

A Blob Container is a folder inside the Storage Account used to organize files (called Blobs : Binary Large Objects). Blobs can be any type of file: CSV, JSON, images, logs, etc.

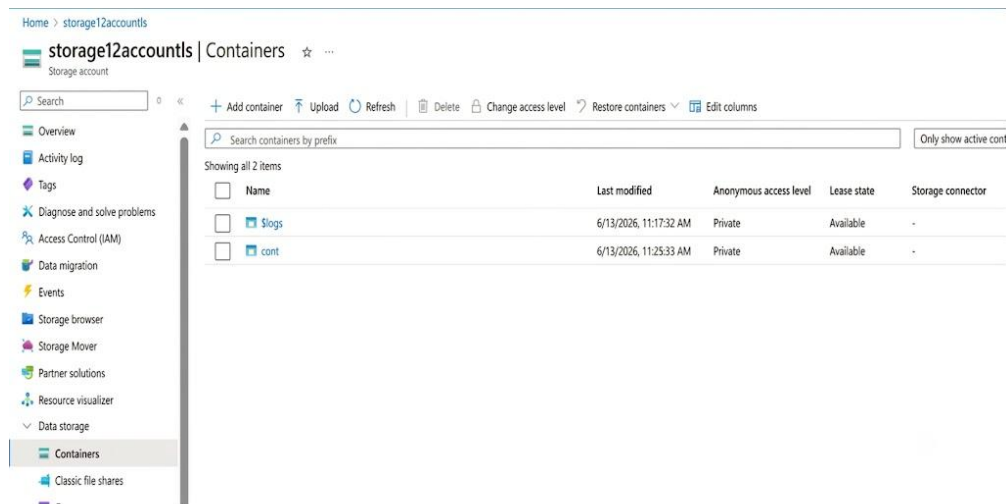
We created container:

- **cont** — the source container where the input CSV file is uploaded

This container setup simulates a real-world data pipeline where raw data arrives at a source location and processed results are saved to a separate destination.

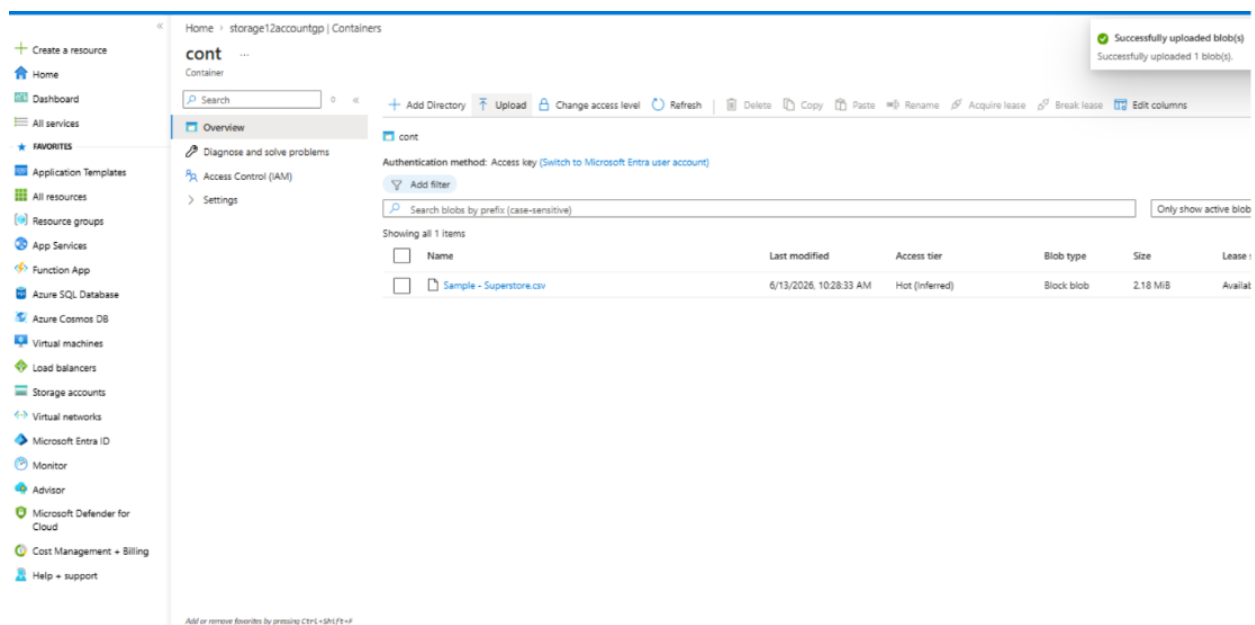


- A container named 'cont' is created successfully and will appear here-



CSV File Upload

The **Sample-Superstore.csv** file was uploaded to the source container as the input data for the pipeline. This file contains sales records and serves as the starting point of the entire data flow. Uploading it manually to Blob Storage simulates how real-world data arrives in a cloud storage system before being processed by a pipeline.

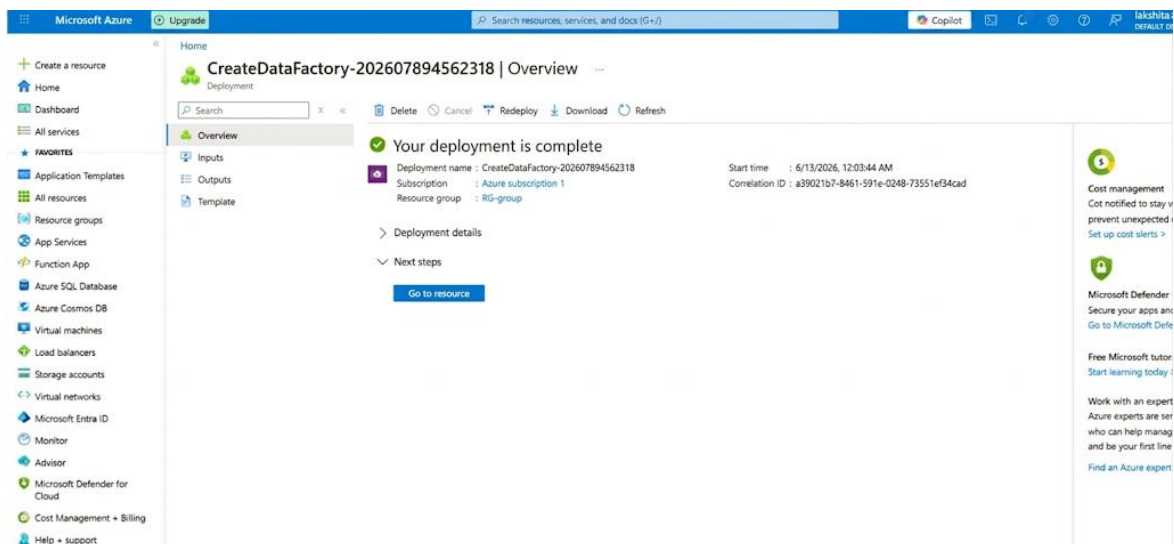
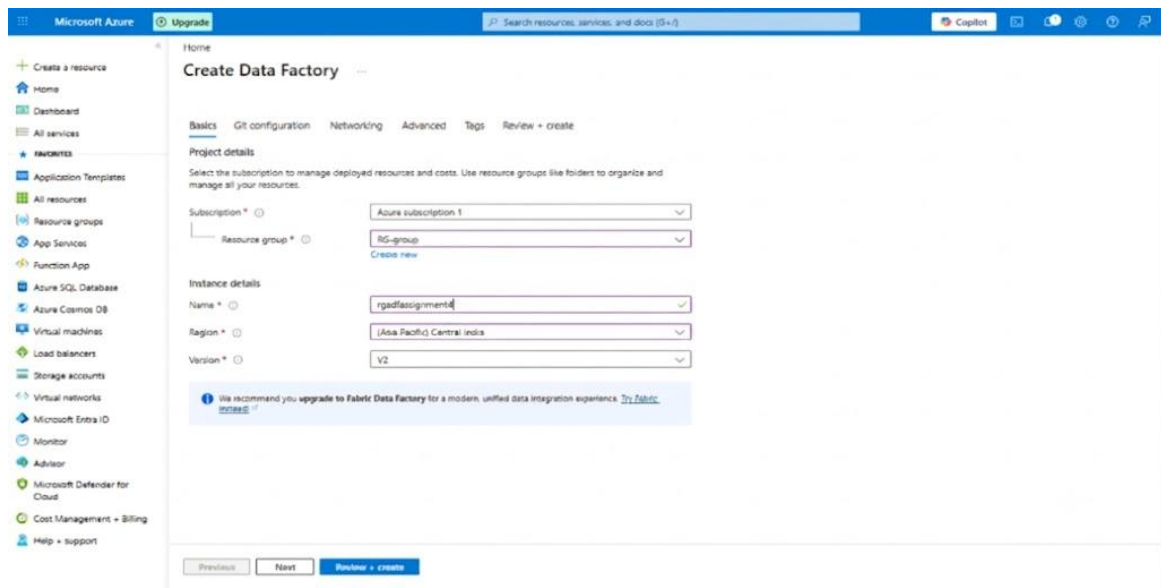


Task 3 -ADF Basics

Azure Data Factory

Azure Data Factory (ADF) is Microsoft's cloud-based data integration and pipeline orchestration service. It allows you to build, schedule, and monitor data workflows that move and transform data between different sources and destinations — all without writing large amounts of code. It is widely used in real-world enterprise data engineering.

We created an ADF instance named **rgadfassignment4** inside the RG-group Resource Group. After deployment, we launched ADF Studio — the visual interface for building and managing pipelines.

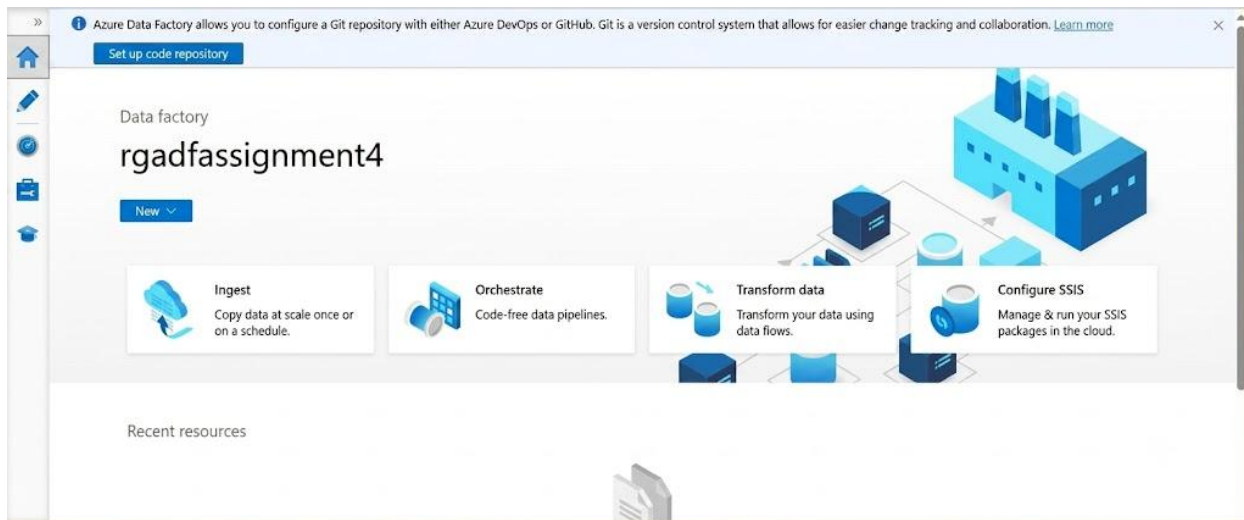


ADF Studio — Three Main Sections :

Author (Pencil Icon) - This is the main workspace where you design and build Pipelines, create Datasets, configure Linked Services, and set up Data Flows. It is where all the development work happens.

Monitor (Graph Icon) - This section provides a real-time view of all Pipeline Runs, Activity Runs, and Trigger Runs. For each run, you can see the execution status (Succeeded, Failed, or In Progress), start time, duration, and detailed logs. It acts as the audit trail for every pipeline execution.

Manage (Wrench Icon) - This section is used to configure backend settings such as Linked Services, Integration Runtimes, Triggers, Global Parameters, and Git Configuration. It is the administrative layer of ADF Studio.



Linked Service (Blob Storage) :

A Linked Service is a saved connection configuration that tells ADF how to authenticate and connect to an external data source or destination. Without a Linked Service, ADF has no way of knowing where your data is or how to access it securely. This Linked Service acts as the secure bridge between ADF and our Blob Storage Account.

Create Linked Service (Azure Blob Storage)

Step 1: Opened Azure Data Factory Studio and navigated to the Manage section (Wrench Icon). Under Linked Services, clicked + New to create a new connection.

Step 2: Searched for Azure Blob Storage, selected it from the available connectors, and clicked Continue.

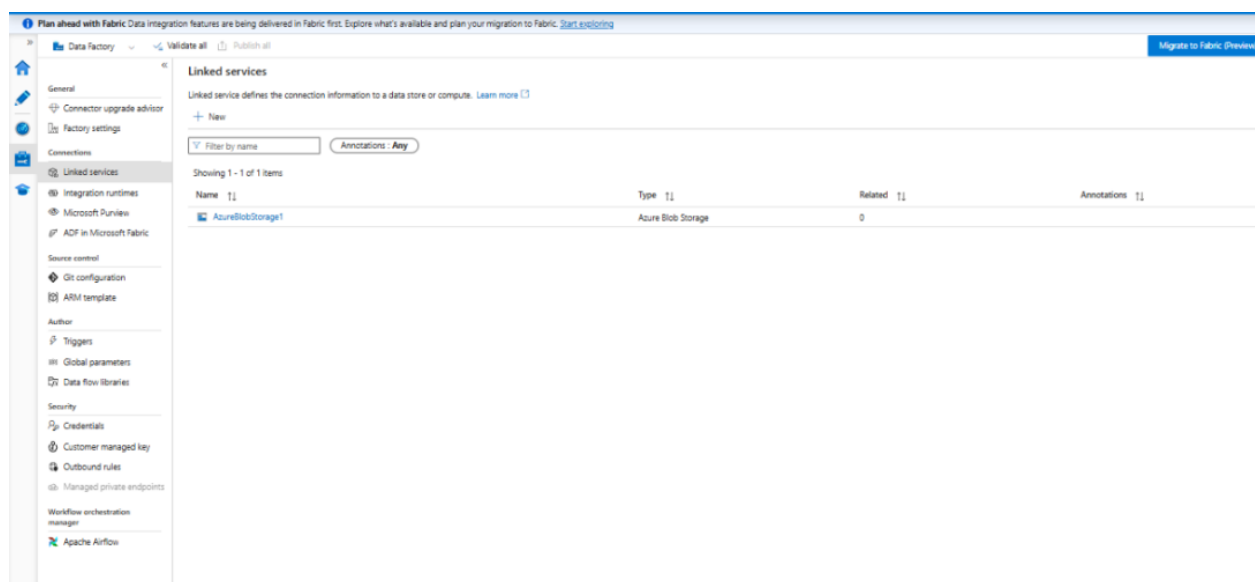
Step 3: Entered a suitable name for the Linked Service and selected Account Key as the authentication type.

Step 4: Chose the appropriate Azure Storage Account that contains the Blob Storage data.

Step 5: Verified the configuration by clicking Test Connection, which confirmed a successful connection.

Step 6: Clicked Create to generate the Linked Service and establish connectivity between Azure Data Factory and Blob Storage.

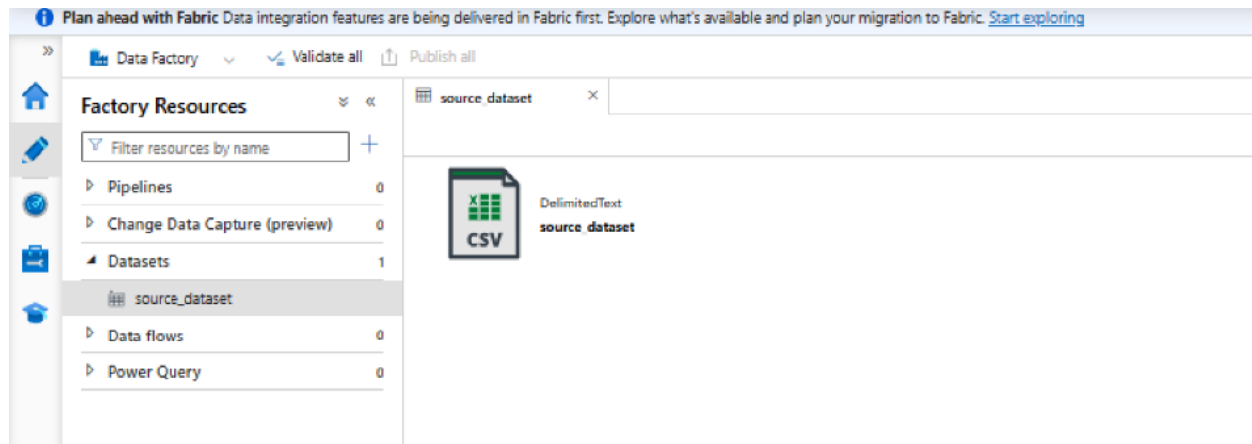
Step 7: Selected Publish All to save and apply the changes to the Data Factory environment.



Datasets (Source + Destination)

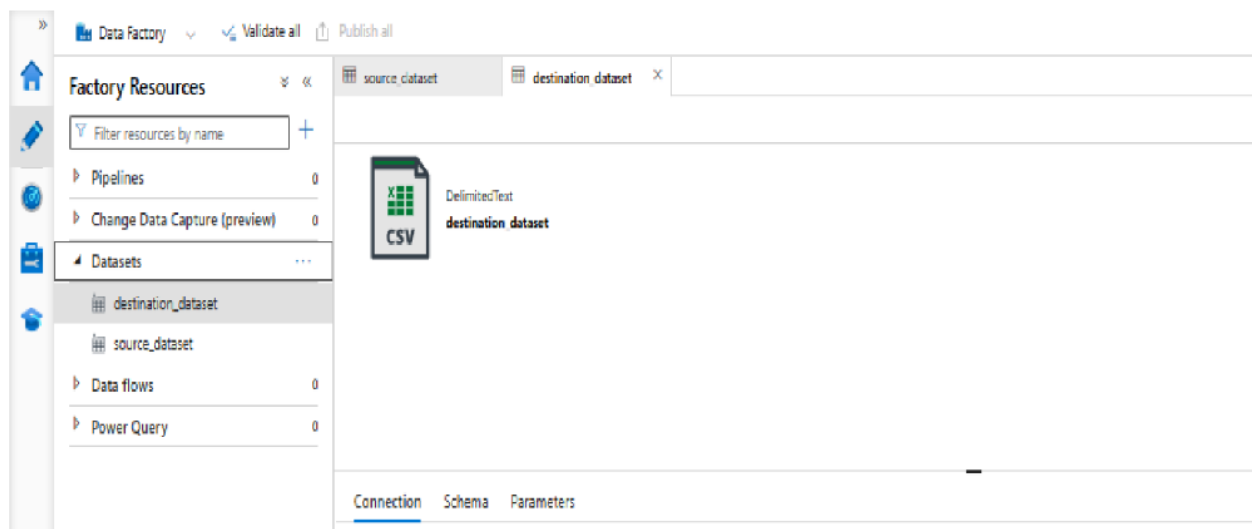
A Dataset in ADF is not the actual data — it is a named pointer that describes where the data is located and what format it is in. Every pipeline activity needs datasets to know what to read from and what to write to. We created two datasets:

- **Source_dataset** - Points to the Sample-Superstore.csv file in the source container. Type: DelimitedText (CSV) with first row as header enabled.



- **Destination_dataset** - Points to the output folder in the destination container. Same format. ADF writes the copied file here after the pipeline runs.

Both datasets are linked to the same **BlobStorageLinkedService** created earlier, which provides the authentication to access both containers.



Get Metadata Activity

The Get Metadata activity is used to read information *about* a file - such as its name (itemName), file type (itemType), and size without actually reading or moving the file content. It is a validation step that confirms the source file exists and is accessible before the pipeline proceeds to copy it.

We added Get Metadata to a new pipeline, connected it to the source_dataset, and configured the field list to retrieve itemName, itemType, and Size. After running Debug, the activity returned

Succeeded with the file details confirmed. This proves the file is present and the connection is working correctly before any data movement begins.

Steps:

Step 1: ADF Studio → Author (Pencil Icon) → Pipelines → + New Pipeline

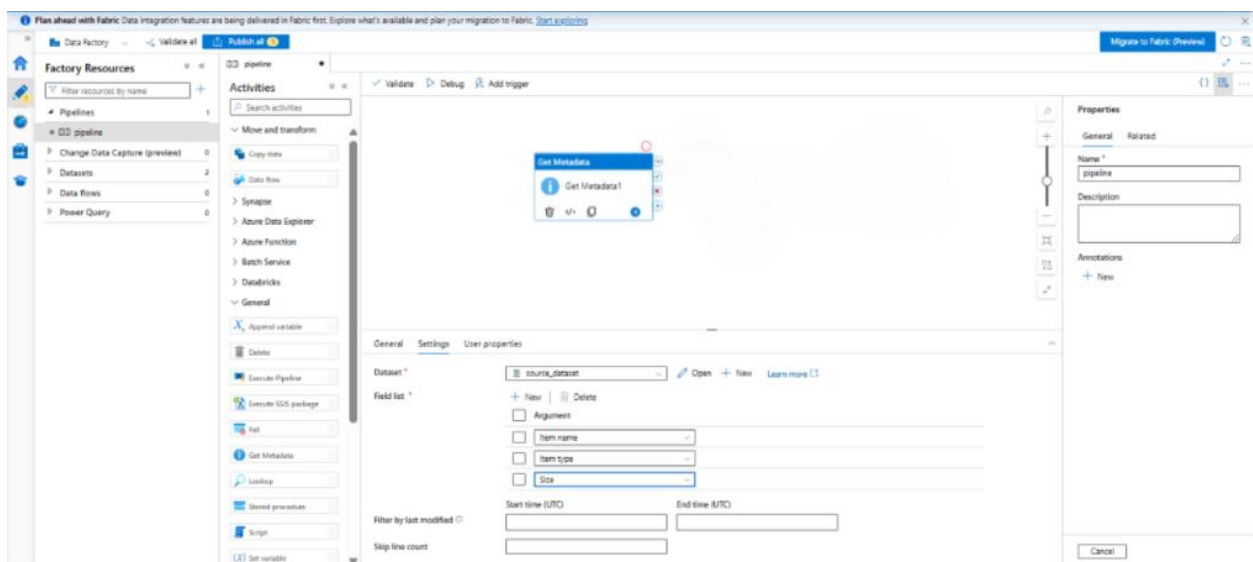
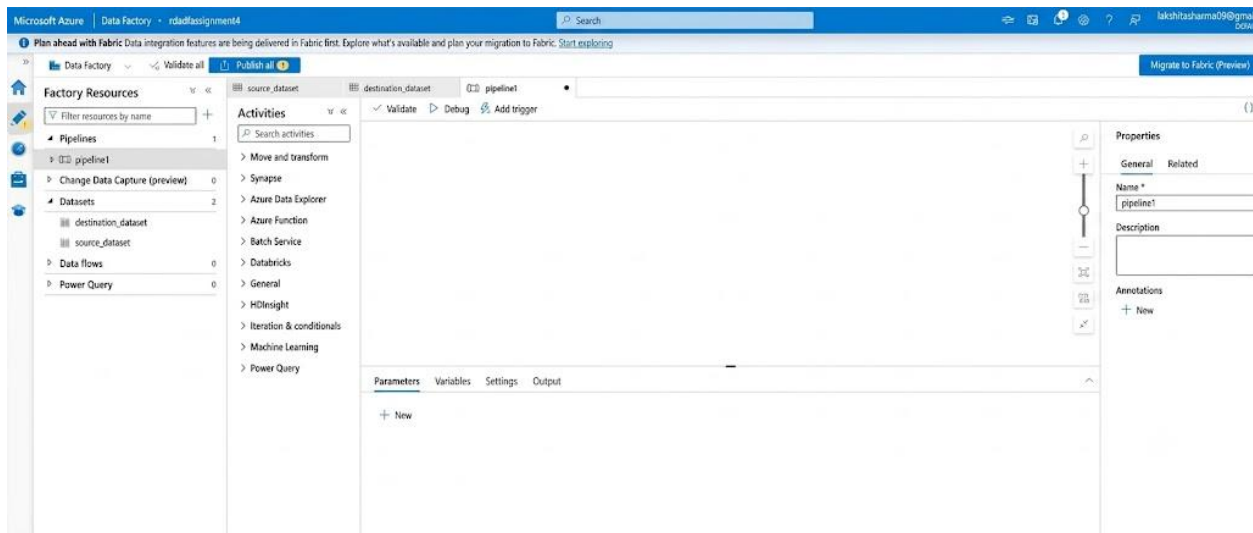
Step 2: Name the pipeline by any name you want.

Step 3: In Activities panel → General → dragged "Get Metadata" onto canvas

Step 4: Clicked on Get Metadata → Settings tab → configure further settings according to the source dataset you had uploaded like item name, type, size.

Step 5: Click debug to test further once it gets succeeded Click Publish All → Publish.

Output: The pipeline is created.



Output- Get metadata1 is successfully uploaded.

Pipeline run ID

Recent 11-2444-2023-b583-921257267471

Pipeline status

Succeeded

Non dating run sub.scription

Succeeded

Monitor in Azure Metrics

Export to CSV

Showing 1 - 1 of 1 items

Activity name	Activity sta...	Activity type	Run start	Duration	Integration runtime	User properties
Get Metadata1	Succeeded	Get Metadata	6/13/2026, 12:10:32 AM	14s	AutoResolveIntegrationRuntime (Central India)	

Task 4 - Pipeline Development

Pipeline in Azure Data Factory

A Pipeline in ADF is a logical grouping of activities that together perform a data processing task. Activities inside a pipeline are connected with arrows that define the order of execution and the conditions under which each activity runs. For example, a Copy Data activity can be configured to run only after a Get Metadata activity succeeds ensuring data validation always happens before data movement.

Step 1: ADF Studio → Author (Pencil Icon) → Pipelines → + New Pipeline

Step 2: Name the pipeline by any name you want. Output- Firstly pipeline is created.

Step 3: Activities panel → Move and Transform → dragged "Copy Data" onto canvas.

Step 4: Click Copy data1 and then set Source and Sink as Source and destination Dataset we had created earlier.

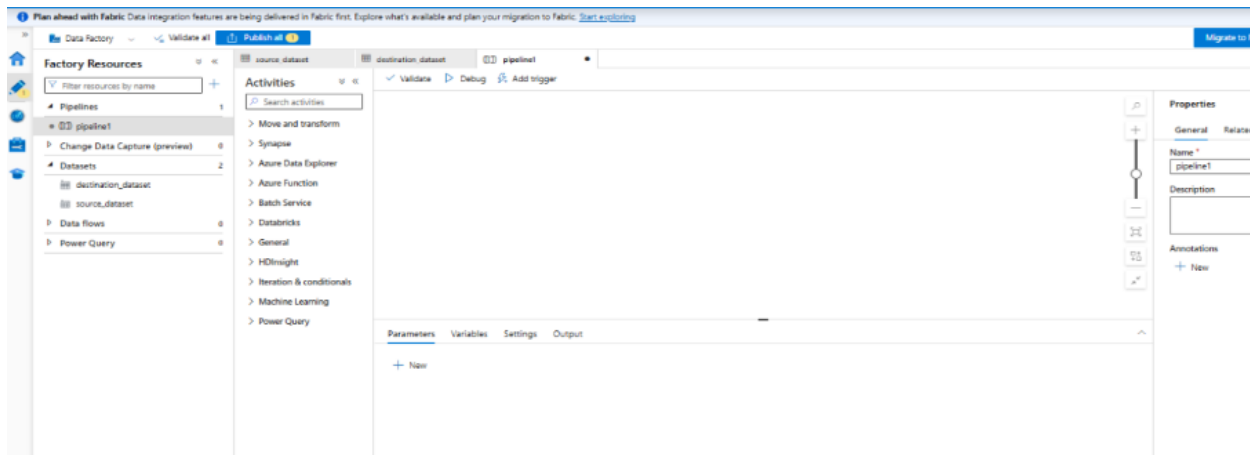
Step 5: Connect Get Metadata1 → Copy Data1 using green arrow.

Copy Data Activity

The Copy Data activity is ADF's primary activity for moving data from one location to another. It reads data from a Source dataset and writes it to a Destination dataset. ADF handles all the underlying data transfer, format handling, and schema mapping automatically.

In this task, we:

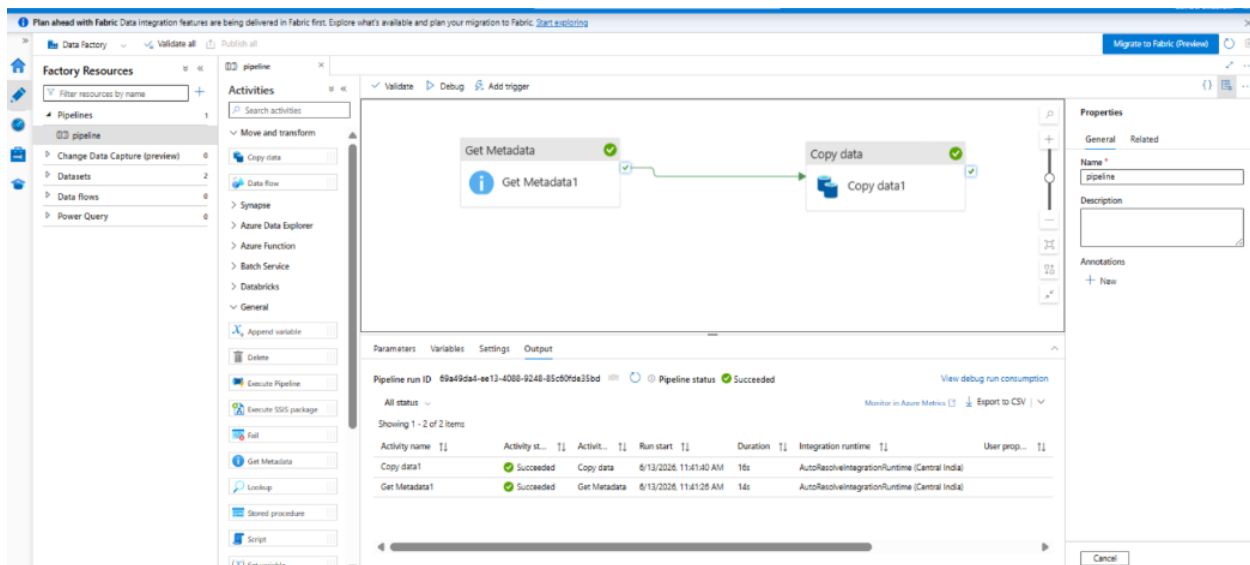
- Added Copy Data to the pipeline canvas from the Move and Transform section of the Activities panel.
- Set the Source to source_dataset (the CSV file in the input container).
- Set the Sink to destination_dataset (the output folder).
- Connected Get Metadata1 → Copy Data1 using a green success arrow, creating a dependency so that Copy Data only runs if the file validation succeeds.



Activity Connection (Control Flow)

The green arrow connecting Get Metadata to Copy Data represents a success dependency. This means ADF will execute Copy Data only if Get Metadata completes without errors. This is a fundamental pipeline design pattern — it prevents the copy operation from running when the source file is missing or inaccessible, avoiding silent failures in production.

After completing the configuration, Publish All was clicked to save the pipeline.



Task 5 - Pipeline Execution

Running a Pipeline : ADF provides two ways to execute a pipeline -

- **Debug Mode** - Runs the pipeline immediately for testing purposes, without requiring a trigger or schedule. It is the standard approach during development to verify that all activities are configured correctly.
- **Trigger (Manual or Scheduled)** - Used in production. A Manual Trigger runs the pipeline on demand, while a Schedule Trigger runs it automatically at defined intervals (hourly, daily, etc.).

In this task, we used both **Debug** and **Manual Trigger** to execute the pipeline multiple times.

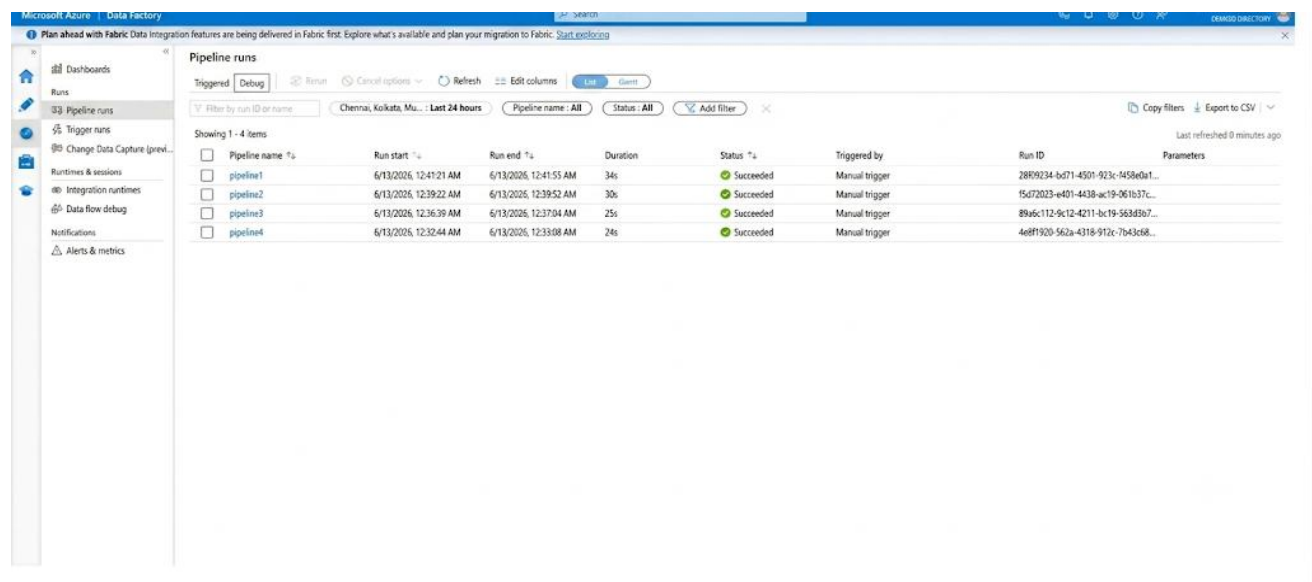
What Happens During Execution

When the pipeline runs, ADF executes each activity in the defined order. On the canvas, each activity displays a real-time status indicator spinning (In Progress), green tick (Succeeded), or red cross (Failed). Clicking the output icon on any activity shows its detailed result, including rows read, rows written, and data size transferred.

Monitor Tab

The Monitor tab records every pipeline run as a permanent log entry. For each run, you can see the pipeline name, start time, end time, duration, trigger type, and status. Drilling into a specific run shows the individual activity results. This is important because it provides traceable evidence that the pipeline ran correctly an essential part of any real data engineering workflow.

In our execution, the pipeline named pipeline was run multiple times and all runs showed Succeeded status, with data successfully copied from the source container to the destination.



The screenshot shows the 'Pipeline runs' section in the Microsoft Azure Data Factory Monitor. The interface includes a left-hand navigation pane with options like Dashboards, Runs, Pipeline runs, Trigger runs, Change Data Capture (prev...), Runtimes & sessions, Integration runtimes, Data flow debug, Notifications, and Alerts & metrics. The main area displays a table of pipeline runs with columns for Pipeline name, Run start, Run end, Duration, Status, Triggered by, and Run ID. The table shows four successful runs for 'pipeline1' through 'pipeline4', all triggered manually. The status for all runs is 'Succeeded' with a green checkmark icon. The table also includes filters for 'Chennai, Kolkata, Mu...' and 'Last 24 hours', and buttons for 'Copy filters' and 'Export to CSV'.

Pipeline name	Run start	Run end	Duration	Status	Triggered by	Run ID
pipeline1	6/13/2026, 12:41:21 AM	6/13/2026, 12:41:55 AM	34s	Succeeded	Manual trigger	28f9234-bd71-4501-923c-5458edat1...
pipeline2	6/13/2026, 12:39:22 AM	6/13/2026, 12:39:52 AM	30s	Succeeded	Manual trigger	15d72023-e401-4438-ac19-061b37c...
pipeline3	6/13/2026, 12:36:39 AM	6/13/2026, 12:37:04 AM	25s	Succeeded	Manual trigger	8946c112-9c12-4211-bc19-563d8b7...
pipeline4	6/13/2026, 12:32:44 AM	6/13/2026, 12:33:08 AM	24s	Succeeded	Manual trigger	4e8f1920-562a-4318-912c-7b43cd8...

Task 6 - IAM Roles

Identity and Access Management (IAM)

IAM in Azure is the permission and security system that controls who can access which resources and what actions they are allowed to perform. Every user, group, or service that interacts with Azure must be granted the appropriate role before they can do anything. This enforces the security principle of *least privilege* — giving each identity only the access it actually needs.

Roles Assigned

Reader - Allows the assigned user to view all resources and their configurations within the defined scope, but does not allow creating, editing, or deleting anything. Useful for giving read-only visibility to team members or auditors.

Contributor - Allows the assigned user to create and manage all types of resources within the scope but does not allow assigning roles to others. Suitable for developers who need to build and configure resources.

Storage Blob Data Contributor - A specialized data-plane role that grants read and write access to the actual blob data (files) inside a Storage Account. This is distinct from general Contributor access it specifically allows reading from and writing to blob containers.

ADF Managed Identity

Azure Data Factory has a Managed Identity an automatically created and managed service account that represents ADF itself in Azure's identity system. Instead of storing credentials or passwords, you assign the Storage Blob Data Contributor role to this Managed Identity, which allows ADF to securely authenticate to the Storage Account and perform read/write operations on blob containers.

In this task, we assigned:

- **Reader** and **Contributor** roles to the user on the Storage Account
- **Storage Blob Data Contributor** role to the ADF Managed Identity on the Storage Account

This configuration ensures that ADF can access the cont (source) container to read the CSV and the output folder to write the result — all without hardcoding any keys or passwords.

Steps:

Step 1: Azure Portal → Storage Accounts → Your account

Step 2: Left sidebar → Access Control (IAM) → Role Assignments tab

Step 3: Clicked + Add → Add Role Assignment → Assign roles: Reader ,Contributor

Step 4: Selected "Reader" → click Next

Step 5: Members → Select Members → select your mail you want to add→ Next → Review + Assign

Step 6: Again Add → Add Role Assignment → Privileged Administrator Roles tab.

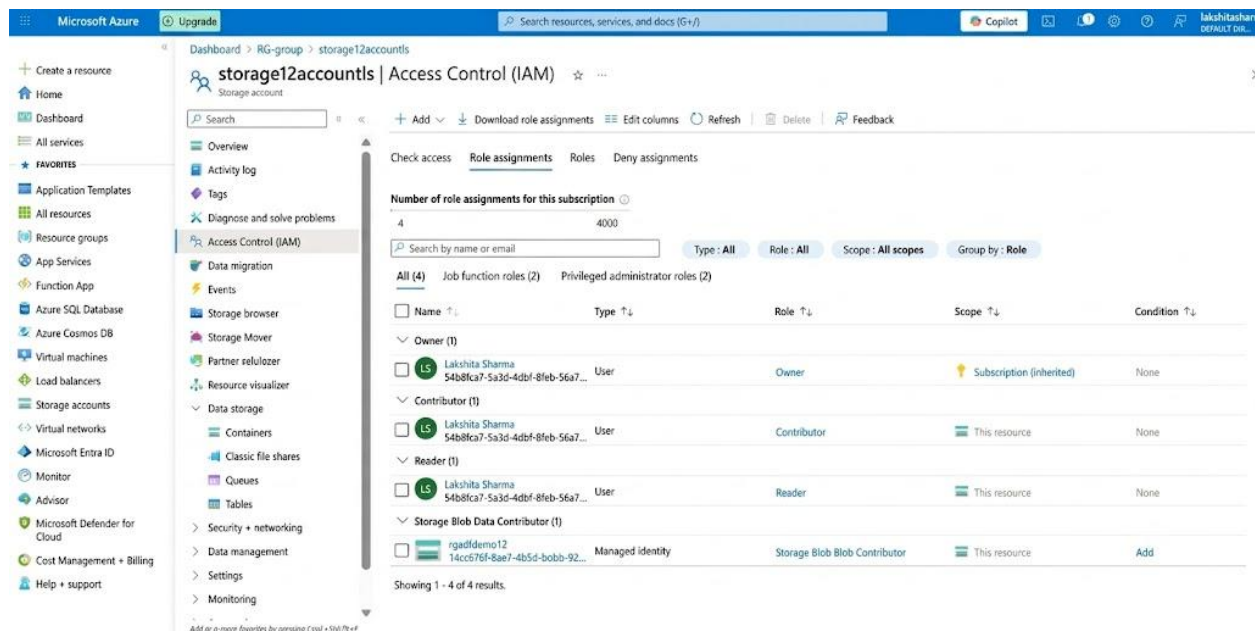
Step 7: Selected "Contributor" → clicked Next

Step 8: Members → Select Members → select your mail you want to add → Review + Assign Provide ADF Access to Storage:

Step 9: Again Add →Add Role Assignment

Step 10: Selected "Storage Blob Data Contributor" → click Next

Step 11: Members → Managed Identity → select your ADF→ Review + Assign



MINI PROJECT

End-to-End Pipeline (Blob → ADF → Destination)

Problem Statement

The Mini Project brings together everything built across Tasks 1 through 6 into a single, complete, working data pipeline. The goal is to read a CSV file from Azure Blob Storage, validate it using metadata, copy it to a new destination, and verify the output using Azure Data Factory as the orchestration engine.

Requirements:

- A CSV file stored in Azure Blob Storage will be used as the source data.

- Configure the connection using a Linked Service and create the required datasets and pipeline in Azure Data Factory.
- Implement data movement through the Copy Data activity and verify file details using the Get Metadata activity.
- Transfer the data to a different file or destination location within the storage environment.

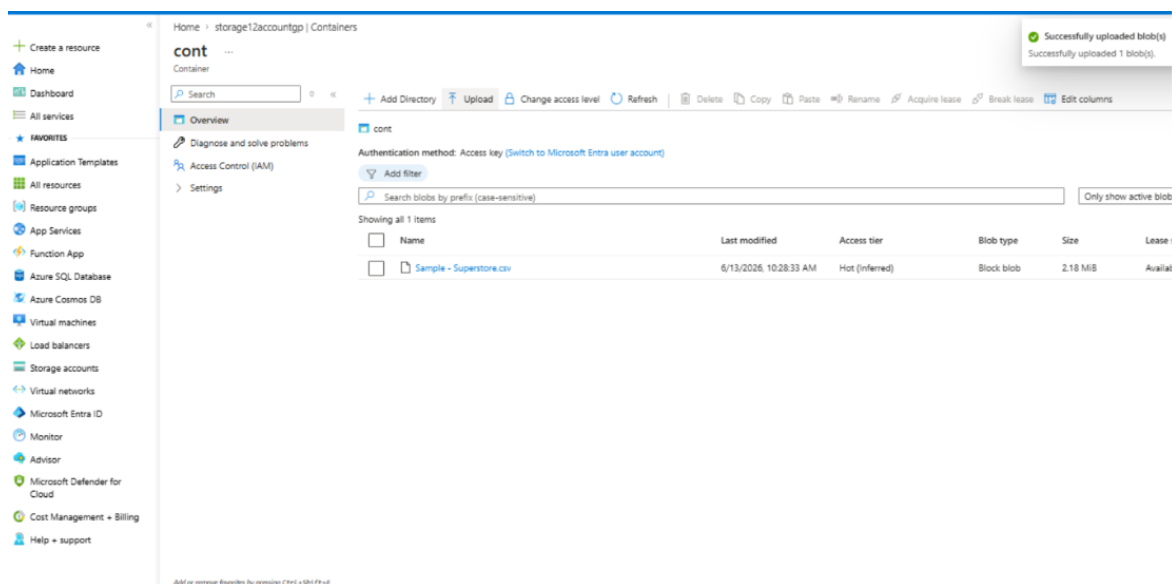
Expected Outcome:

- Successful execution of the Azure Data Factory pipeline.
- Source data accurately copied to the specified destination.
- File metadata retrieved and validated to ensure data integrity and proper pipeline operation.

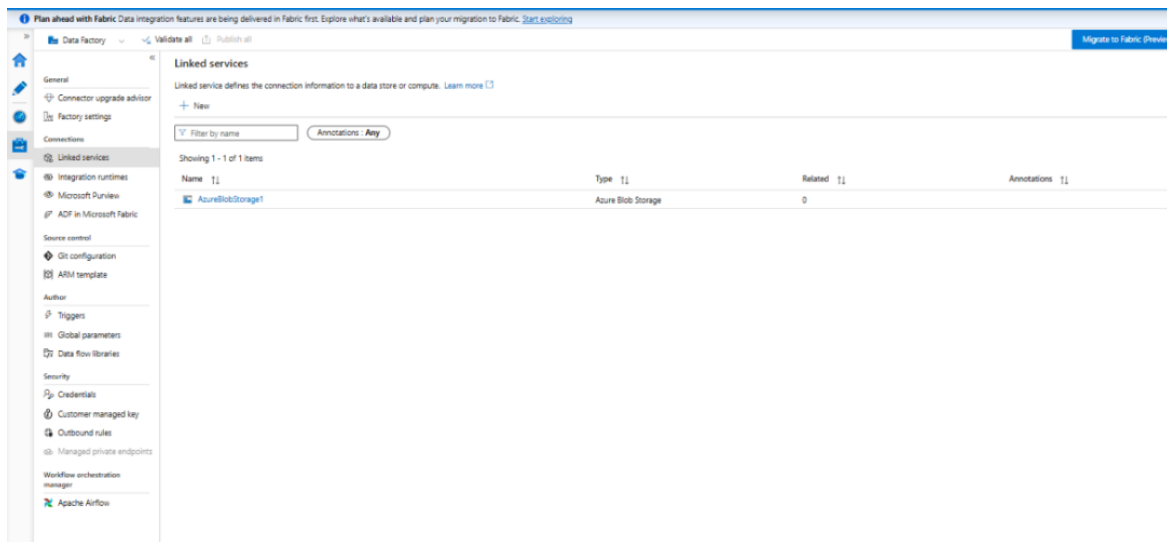
What Was Built

The complete pipeline flow is:

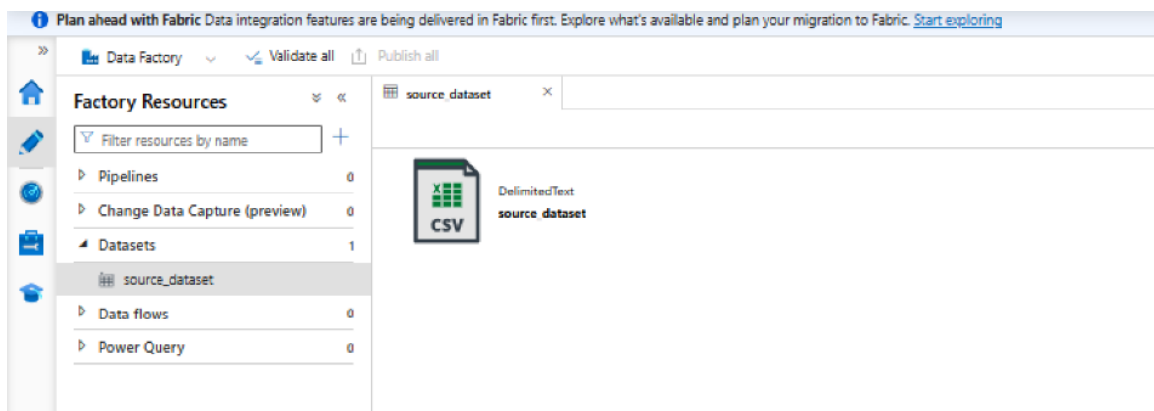
1. **Source** → The Sample-Superstore.csv file stored in the cont Blob Container serves as the raw input data.



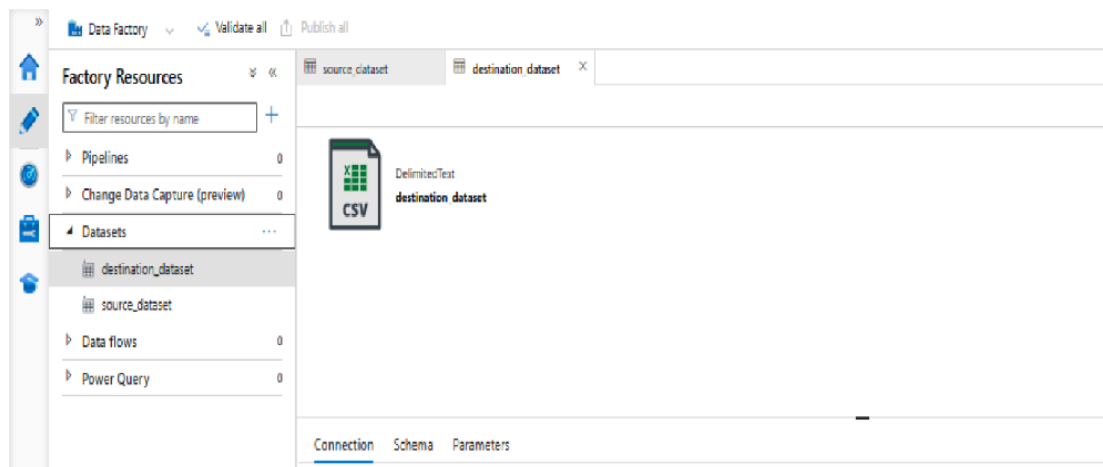
2. **Linked Service** → The AzureBlobStorage1 Linked Service provides ADF with authenticated access to the Storage Account using Account Key, establishing a secure connection.



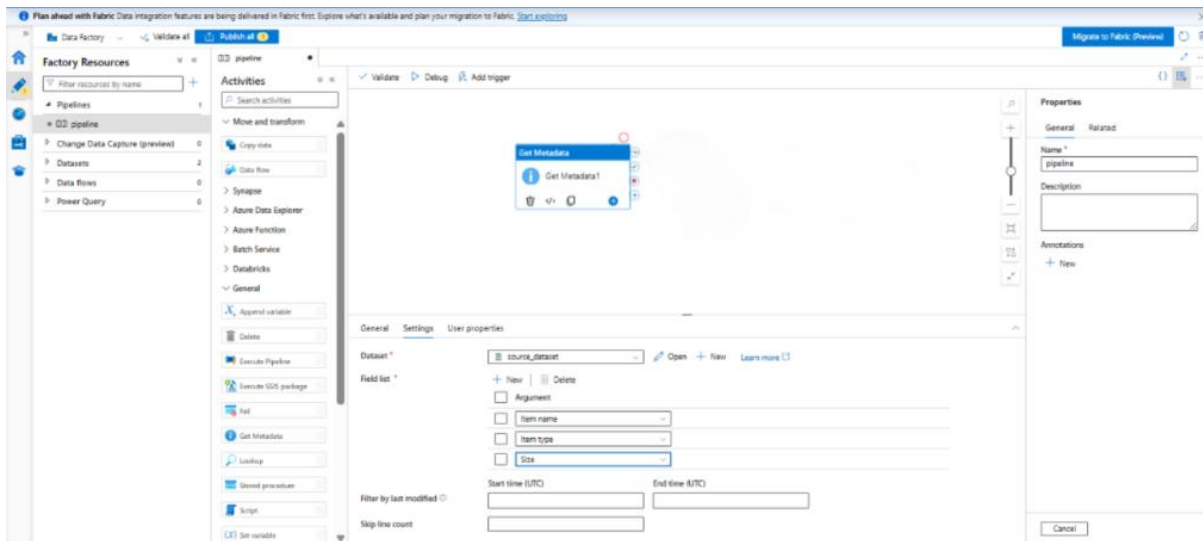
3. **Source Dataset** → source_dataset points to the source CSV file in the input container.



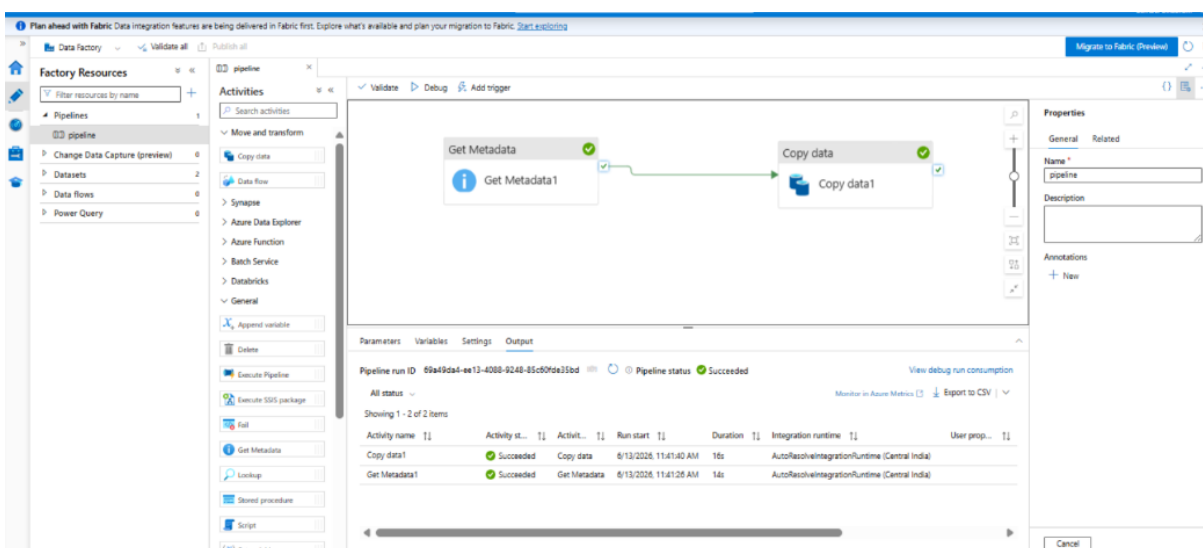
4. **Destination Dataset** → destination_dataset points to the cont/output/ folder. This is where ADF writes the copied file.



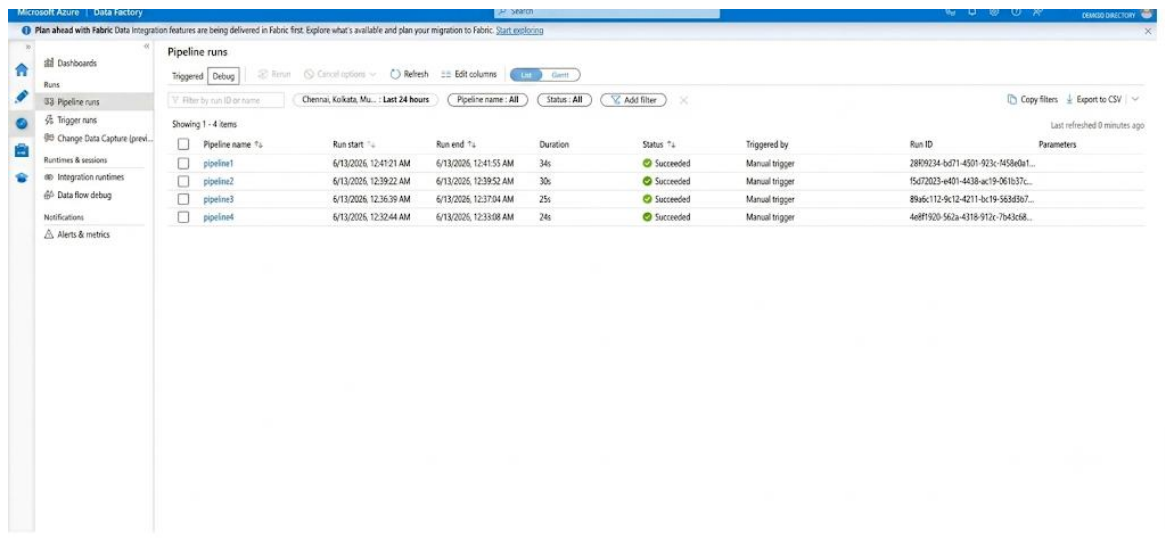
5. **Get Metadata Activity** → Configured with field list: itemName, itemType, and Size. This activity runs first and validates that the source file actually exists and retrieves its properties before any data movement begins. If the file is missing, the pipeline stops here preventing unnecessary operations.



6. **Copy Data Activity** → Connected to Get Metadata via a success arrow. Reads from source_dataset and writes to destination_dataset. Executed only after metadata validation passes successfully.



7. Monitor section showing all pipeline runs with Succeeded status confirming successful pipeline execution.

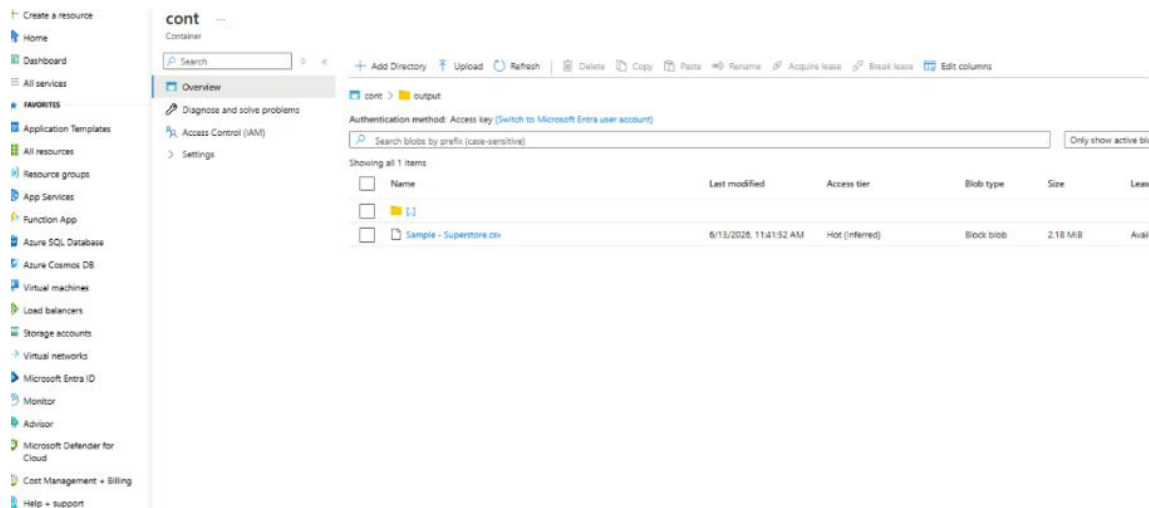


The screenshot shows the 'Pipeline runs' section in the Microsoft Azure Data Factory Monitor. The interface includes a left-hand navigation pane with options like Dashboards, Runs, Pipeline runs, Trigger runs, Change Data Capture, Runtime & sessions, Integration runtimes, Data flow debug, Notifications, and Alerts & metrics. The main area displays a table of pipeline runs. The table has columns for Pipeline name, Run start, Run end, Duration, Status, Triggered by, Run ID, and Parameters. All four listed runs (pipeline1, pipeline2, pipeline3, pipeline4) are in a 'Succeeded' status, triggered by a 'Manual trigger'.

Pipeline name	Run start	Run end	Duration	Status	Triggered by	Run ID	Parameters
pipeline1	6/13/2025, 12:41:21 AM	6/13/2025, 12:41:55 AM	34s	Succeeded	Manual trigger	28f9234-bd71-4501-923c-4d58da1...	
pipeline2	6/13/2025, 12:39:22 AM	6/13/2025, 12:39:52 AM	30s	Succeeded	Manual trigger	15d72023-e401-4438-ac19-061637c...	
pipeline3	6/13/2025, 12:36:39 AM	6/13/2025, 12:37:04 AM	25s	Succeeded	Manual trigger	8b46c112-9c12-4211-bc19-563d367...	
pipeline4	6/13/2025, 12:32:44 AM	6/13/2025, 12:33:08 AM	24s	Succeeded	Manual trigger	4edf920-562a-4318-912c-7b43c68...	

Results and Verification

The pipeline was executed successfully. Both activities **Get Metadata1** and **Copy Data1** showed **Succeeded** status in the pipeline output. The Monitor tab confirmed multiple successful runs with consistent execution. The final verification showed that Sample-Superstore.csv was successfully copied to the `cont/output/` folder in Blob Storage, confirming that data moved correctly from source to destination.



The screenshot shows the Microsoft Azure Storage Explorer interface. The left-hand navigation pane lists various Azure services, including Home, Dashboard, All services, Application Templates, All resources, Resource groups, App Services, Function App, Azure SQL Database, Azure Cosmos DB, Virtual machines, Load balancers, Storage accounts, Virtual networks, Microsoft Entra ID, Monitor, Advisor, Microsoft Defender for Cloud, and Cost Management + Billing. The main area displays the 'Overview' tab for a container named 'cont'. The container's contents are listed in a table with columns for Name, Last modified, Access tier, Blob type, Size, and Lease. The table shows a single item named 'Sample - Superstore.csv' with a last modified date of 6/13/2025, 11:41:52 AM, an access tier of 'Hot (Inferred)', a blob type of 'Block blob', a size of 2.18 MB, and a lease status of 'Avail'.

Name	Last modified	Access tier	Blob type	Size	Lease
Sample - Superstore.csv	6/13/2025, 11:41:52 AM	Hot (Inferred)	Block blob	2.18 MB	Avail

This mini project demonstrates a real-world data pipeline pattern used daily in enterprise data engineering. In production, pipelines like this move data from landing zones into data warehouses, databases, or analytics platforms automatically, securely, and with built-in validation.